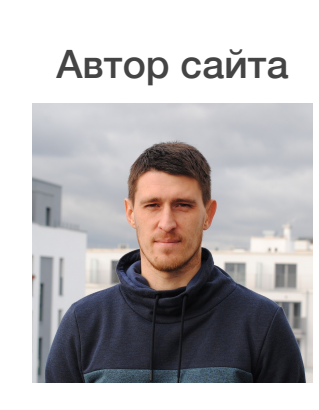
Language

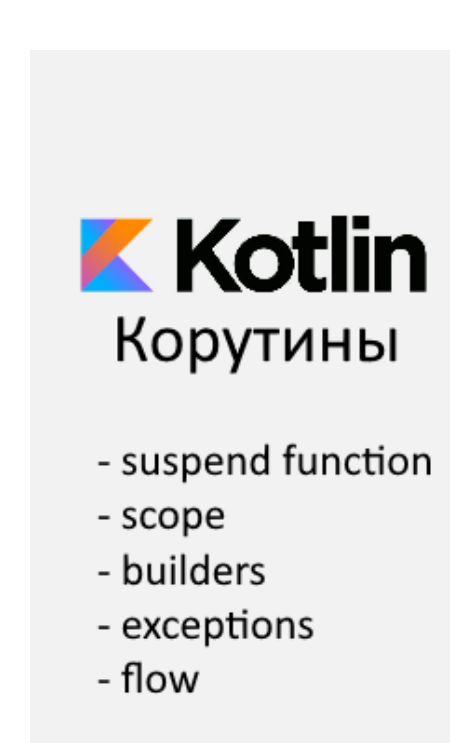
Автор сайта



Дмитрий Виноградов

Подробнее можно посмотреть или почитать.

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.



Kotlin
Корутины
- suspend function
- scope
- builders
- exceptions
- flow

Социальные сети

В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте

StartAndroid
15 067 подписчиков
Подписаться

Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551306702056

Paypal

Урок 35. User Input. PointerEvent. PointerInputChange.

28 февраля 2024

В этом уроке получаем события в Modifier pointerInput. Используем функции awaitPointerEventScope и awaitPointerEvent. Разбираемся, какая информация о касаниях нам доступна.

У жестов в Compose есть несколько API уровней. Похожее разделение было и у анимации.

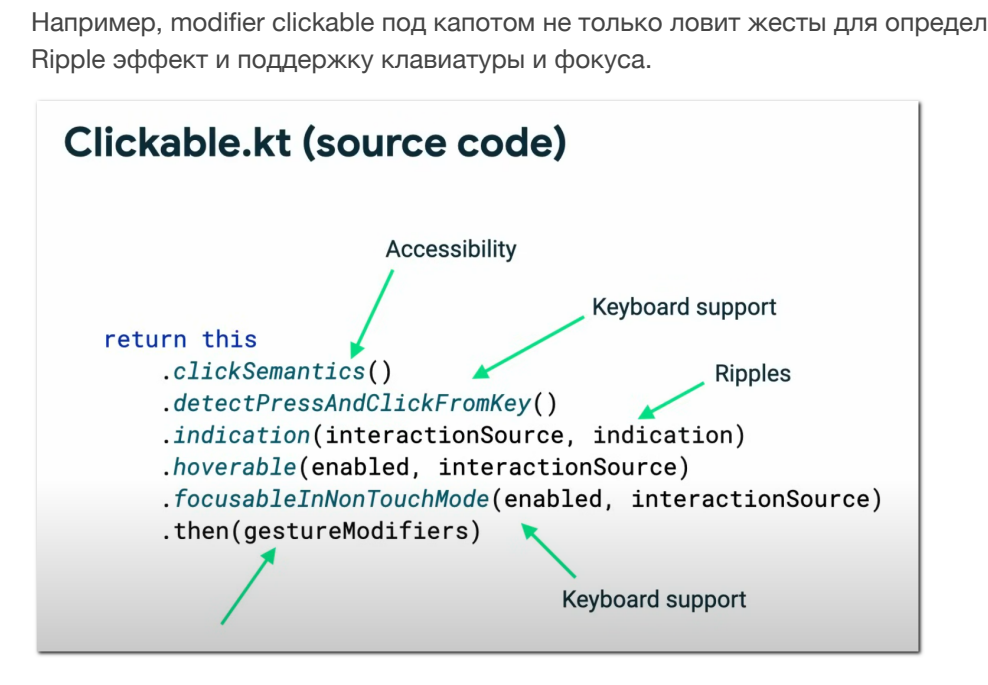
Верхний уровень - это существующие UI элементы. Например, Button сразу предоставляет нам onClick лямбду, а LazyColumn поддерживает скролл.

Средний уровень - Modifiers. С ними легко можно добавлять жесты UI элементам, которые по умолчанию жесты не поддерживают. Например - использовать clickable, чтобы добавить клики в Text или Box.

Низкоуровневую работу предоставляет нам modifier pointerInput. В нем мы можем использовать detect* и await* операторы, чтобы работать с жестами (Tap, Drag) или отдельными Pointer-событиями (Press, Move, Released).

Когда возникает вопрос: какой уровень выбрать, начинаем искать подходящее решение сверху вниз. Чем выше уровень - тем решение будет проще и надежнее, но функционал будет ограничен. Чем ниже уровень, тем решение будет гибче, но сложнее и более подвержено ошибкам, т.к. мы сами его создаем.

Например, modifier clickable под капотом не только ловит жесты для определения клика, но и добавляет инфу для Accessibility, Ripple эффект и поддержку клавиатуры и фокуса.



Clickable.kt (source code)

```
return this
    .clickSemantics()
    .detectPressAndClickFromKey()
    .indication(interactionSource, indication)
    .hoverable(enabled, interactionSource)
    .focusableInNonTouchMode(enabled, interactionSource)
    .then(gestureModifiers)
```

Источник

Решение идеально подходит, чтобы добавить возможность ловить клики. Но он не предоставляет информацию о координатах клика. Если нам необходимо получать такие данные, то придется использовать низкоуровневое решение.

pointerInput

Как и в уроках про анимацию, начнем изучение с низкоуровневых решений и будем подниматься выше, чтобы лучше все усвоить и понимать используемые исходники.

В качестве примера используем экран:

```
@Composable
fun HomeScreen() {
   val logs = remember { mutableStateListOf<String>() }
   Column {
       Box(modifier = Modifier
           .size(300.dp)
           .background(Color.LightGray)
       )
       Text(text = logs.takeLast(15).joinToString("\n"), fontSize = 14.sp)
   }
}
```

В нем пустой Box, который будет полигоном для жестов. И Text, который будет отображать логи: последние 15 элементов из mutable коллекции logs.

Чтобы научить Box работать с касаниями и жестами, добавим ему Modifier pointerInput.

```
@Composable
fun HomeScreen() {
   val logs = remember { mutableStateListOf<String>() }
   Column {
       Box(modifier = Modifier
           .size(300.dp)
           .background(Color.LightGray)
           .pointerInput(Unit) {

           }
       )
       Text(text = logs.takeLast(15).joinToString("\n"), fontSize = 14.sp)
   }
}
```

Блок кода в pointerInput не является бесконечным циклом, куда валятся все input события. Этот блок кода будет выполнен один раз, в момент первого события, например - первого касания. Наша задача тут - запустить механизм, который будет ловить все последующие события.

Для этого мы стартуем специальную корутину awaitPointerEventScope:

```
.pointerInput(Unit) {
   awaitPointerEventScope {


   }
}
```

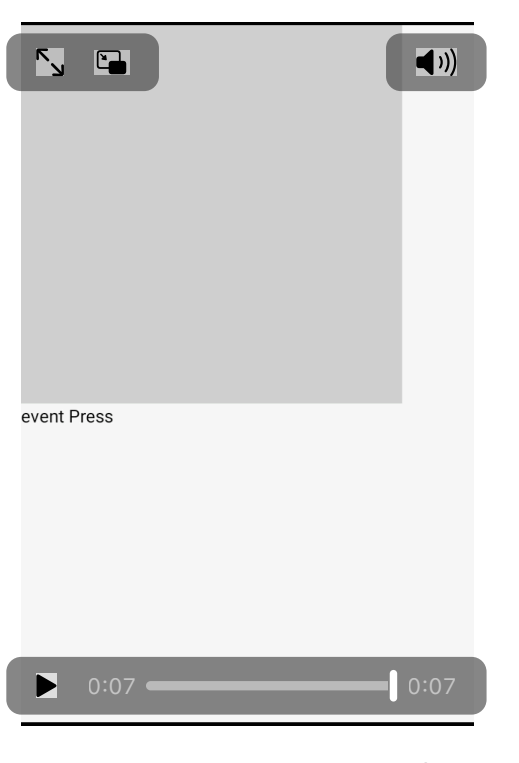
В эту корутину будут прилетать события, а мы их сможем ловить с помощью специальных suspend функций.

Основная из них - это awaitPointerEvent:

```
.pointerInput(Unit) {
   awaitPointerEventScope {
       val event = awaitPointerEvent()
       logs.add("event ${event.type}")
   }
}
```

awaitPointerEvent возвращает нам пойманное событие PointerEvent, и мы выводим в лог его тип.

Запускаем и кликаем:



event Press

Первое нажатие выводит в лог событие Press. Но последующие нажатия ни к чему не приводят. Чтобы понять почему, надо немного погрузиться в детали и понять, что произошло под капотом.

Вызов awaitPointerEventScope запускает корутину, которая добавляет себя в список слушателей input событий.

awaitPointerEvent - это suspend функция. Как мы знаем, suspend функции на время выполнения своей задачи приостанавливают выполнение кода корутины. Такой задачей может быть просто delay, запрос в БД или к серверу и т.п. Как только задача выполнена и получен результат, suspend функция возобновляет выполнение корутины.

Задачей awaitPointerEvent является ничего-не-делание, пока не придет событие. Т.е. awaitPointerEvent приходит к корутине awaitPointerEventScope и говорит примерно следующее: “Ты там подписалась на получение событий. Я тогда тут посижу, подожду. Как только событие придет, давай его мне. Я верну его, как результат своей работы и возобновлю выполнение твоего кода.”

Когда мы нажали на Box первый раз, случилось следующее.

Выполнился код в pointerInput. Напомню, что он выполняется только при первом событии. Это что-то типа инициализации. Корутина awaitPointerEventScope запустилась и подписалась на события.

Всем слушателям было отправлено событие Press. Корутина awaitPointerEventScope получила это событие и передала его в awaitPointerEvent, который вернул его, как результат своей работы, и возобновил выполнение кода в awaitPointerEventScope.

Далее выполнился наш код логирования. На этом корутина awaitPointerEventScope завершилась и удалила себя из списка слушателей событий.

Все последующие нажатия НЕ запускают снова код в pointerInput, а только отправляют события. Но т.к. корутина awaitPointerEventScope завершила свою работу, то она больше не реагирует на эти события.

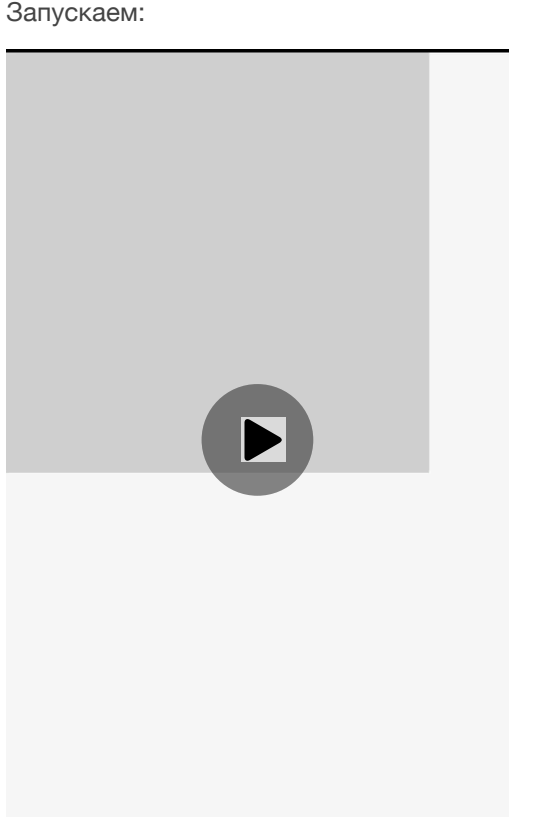
Наша задача - создать в pointerInput такой механизм, который запустится один раз, но будет ловить все последующие события.

Для этого нам необходимо запустить бесконечный цикл.

```
.pointerInput(Unit) {
   awaitPointerEventScope {
       while(true) {
           val event = awaitPointerEvent()
           logs.add("event ${event.type}")
       }
   }
}
```

Помещаем вызов awaitPointerEvent в цикл. Это приведет к тому, что получив событие, awaitPointerEvent вернет его нам, мы отправим в лог его тип, а awaitPointerEvent снова приостановит корутину в ожидании следующего события и т.д.

Запускаем:



Теперь видно, что после первого события Press (нажать) идет событие Release (отпустить). А если подвигать нажатым пальцем, то между Press и Release получим пачку Move. Это уже похоже на классическую схему получения input событий.

changes

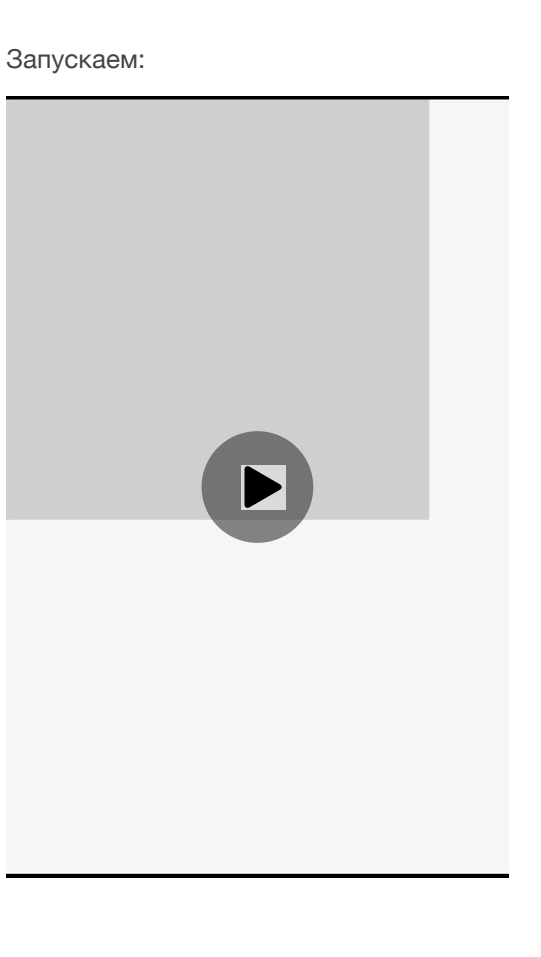
Чего нам теперь не хватает, так это координат - т.е. в каком именно месте Box произошло нажатие.

Эти данные мы можем найти в PointerEvent.changes. Это список объектов PointerInputChange. Пока у нас только одно касание, в этом списке находится только один элемент. И он содержит в себе данные о касании.

В списке changes может быть несколько элементов, если использовать мультитач. Мы же пока просто будем брать первый элемент из списка и логировать координаты и состояние нажатия:

```
.pointerInput(Unit) {
   awaitPointerEventScope {
       while(true) {
           val event = awaitPointerEvent()
           val change = event.changes[0]
           logs.add("${event.type} ${change.position} ${change.pressed} ")
       }
   }
}
```

Запускаем:



Кроме Press и Release мы можем получить события Enter и Exit. Для этого надо запустить приложение не на эмуляторе, а на девайсе. Студия отображает его экран, а я просто двигаю мышкой. Когда мышь заходит в область Box, мы получаем событие Enter. Далее идет куча Move событий. А в момент выход за пределы Box получаем Exit.

И обратите внимание, что pressed = false для всех этих событий, потому что я не нажимал кнопки мыши.

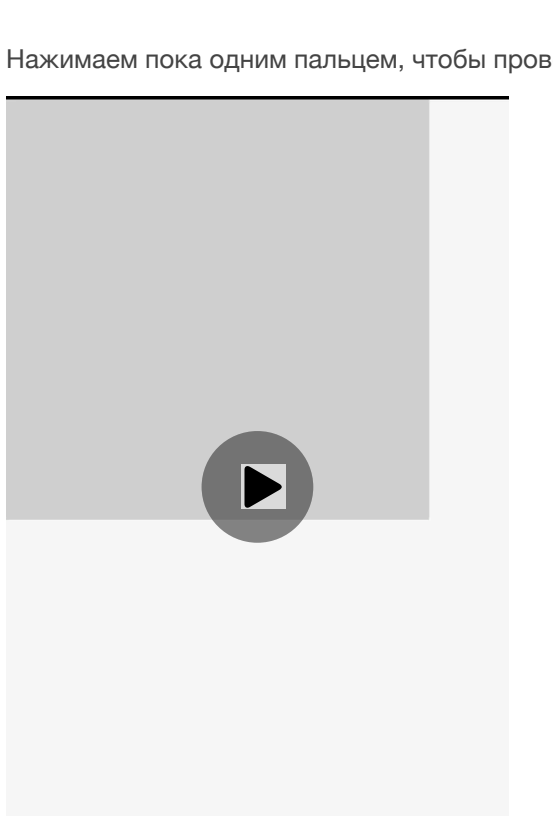
PointerInputChange также содержит в себе информацию о касании во время предыдущего события. Например, поле previousPosition. При движении пальца по экрану мы легко можем рассчитать, на сколько сдвинулся палец по сравнению с прошлым Move.

Multitouch

Теперь давайте посмотрим на работу мультитач. Будем выводить в логи инфу о всех текущих касаниях. Для этого будем брать все элементы списка changes и для каждого из них выводить его ID и состояние нажатия.

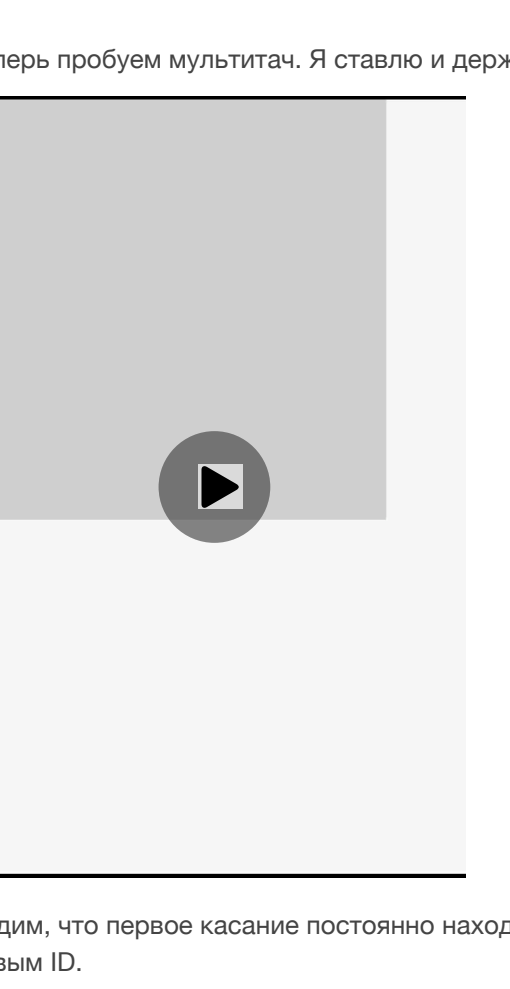
```
.pointerInput(Unit) {
   awaitPointerEventScope {
       while(true) {
           val event = awaitPointerEvent()
           event.calculateZoom()
           val changes = event.changes.map { "${it.id.value} ${it.pressed}" }
           logs.add("${event.type} $changes")
       }
   }
}
```

Нажимаем пока одним пальцем, чтобы проверить логи:



Обратите внимание, что каждый новый тач получает свой ID.

Теперь пробуем мультитач. Я ставлю и держу один палец, а другим делаю простые клики.

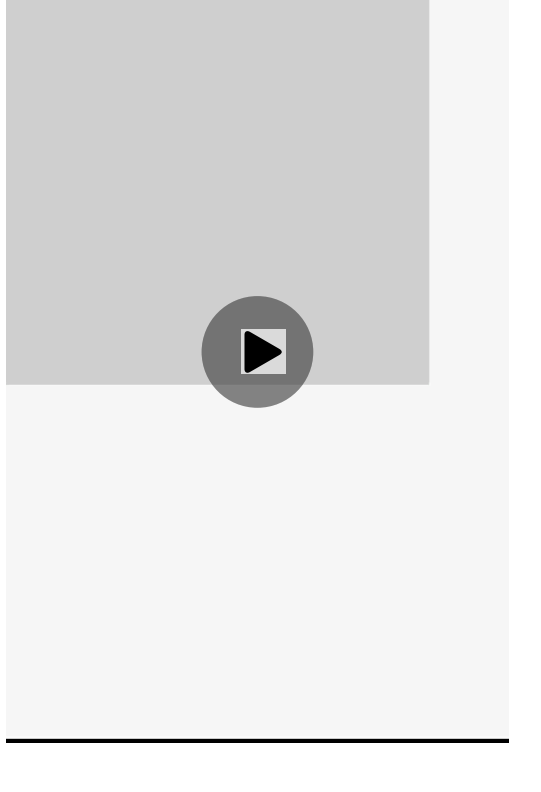


Видим, что первое касание постоянно находится в списке, не меняя ID. А второе добавляется и удаляется, каждый раз с новым ID.

Обратите внимание, что после того как я убираю второй палец (Release), всегда приходит Move. И это не у меня такая рука дерганная. Это скорее похоже на какой-то лайфхак системы, чтобы последнее полученное событие содержало актуальное количество касаний.

Т.е. когда я убираю второй палец, приходит событие Release, которое содержит инфу еще о двух касаниях (одно pressed, а другое - нет). Но по факту остается лишь одно. И последующий Move уже приносит нам обновленный список changes - с одним касанием.

Если вторым пальцем не просто кликать, а делать при этом небольшие передвижения, то получаем кучу Move, каждый из которых содержит всю инфу о касаниях



Попробуем поставить и убрать 4 пальца, один за другим

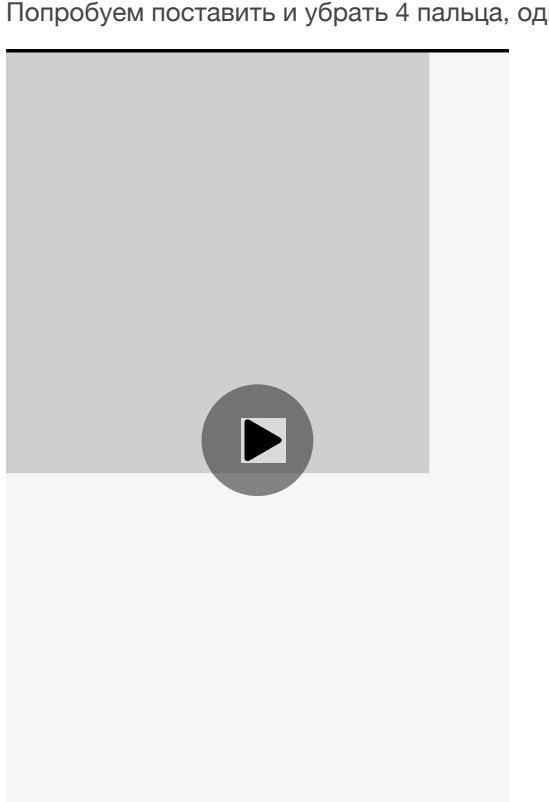
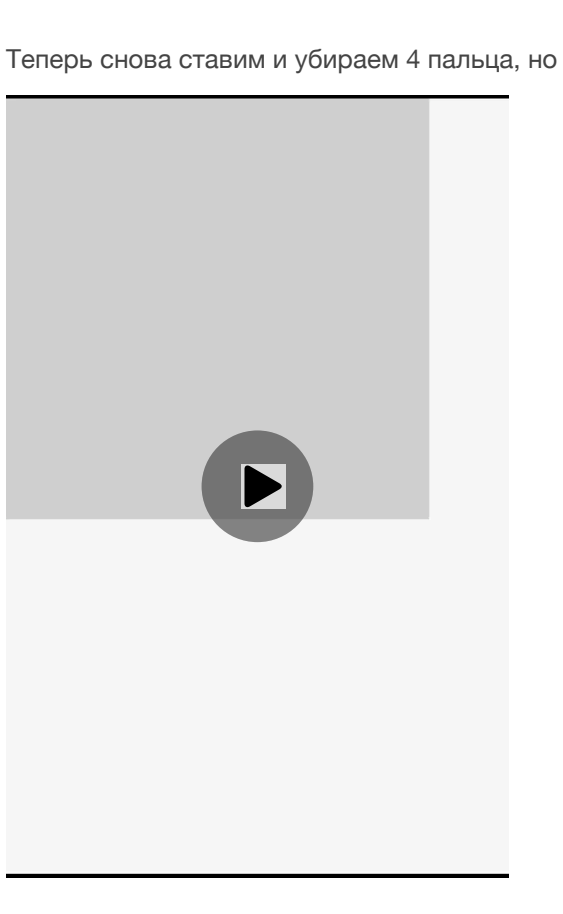


Схема та же. После каждого Release почему-то прилетает Move.

Теперь снова ставим и убираем 4 пальца, но одновременно



А вот тут уже Move после Release не появился. У меня нет объяснения, почему так происходит, просто знайте это и учитывайте в реализуемой тач-логике.

calculate

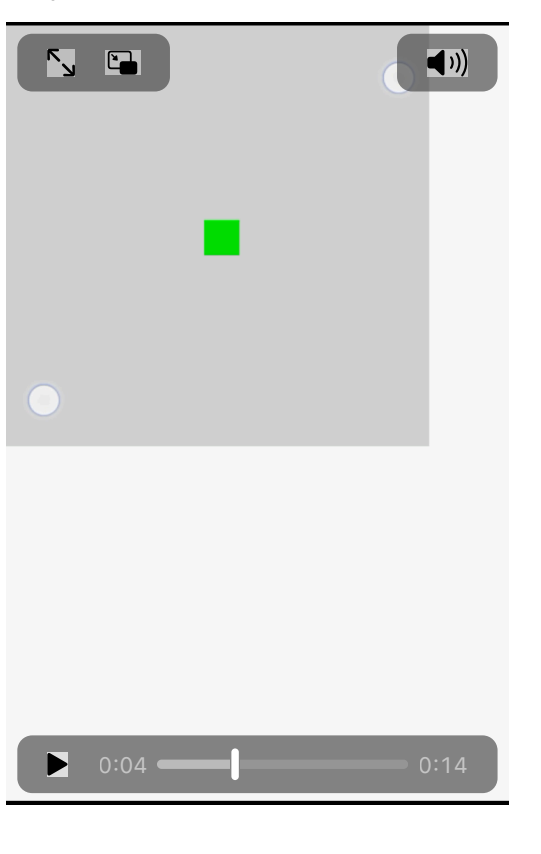
У PointerEvent есть методы, которые позволяют получить полезные значения на основе данных из changes. Вы можете открыть исходники и посмотреть, как они работают, чтобы создавать свои похожие методы.

calculateCentroid - вычисляет координаты центра между всеми касаниями.

Чтобы было нагляднее, я буду эти координаты выводить не в лог, а рисовать по ним квадрат. Для этого создаем новый State с типом Offset. Туда пишем данные о координатах Centroid и поверх Box рисуем квадрат по этим координатам.

```
@Composable
fun HomeScreen() {
   val point = remember { mutableStateOf(Offset.Zero) }
   Column {
       Box(modifier = Modifier
           .size(300.dp)
           .background(Color.LightGray)
           .pointerInput(Unit) {
               awaitPointerEventScope {
                   while (true) {
                       val event = awaitPointerEvent()
                       val centroid = event.calculateCentroid()
                       point.value = centroid.takeIf { it != Offset.Unspecified } ?: Offset.Zero
                   }
               }
           }
           .drawWithContent {
               drawContent()
               drawPoints(listOf(point.value), PointMode.Points, Color.Green, 50f)
           }
       )
   }
}
```

Результат:



По Centroid есть еще методы:
calculateCentroidSize - среднее расстояние от Centroid до точек касания
calculatePan - насколько Centroid смещен по сравнению со своим предыдущим положением

И напоследок рассмотрим пару интересных calculate методов:
calculateZoom - изменение размера по сравнению с предыдущим положением
calculateRotation - градус поворота по сравнению с предыдущим положением

Эти методы можно добавить в пример с квадратом.

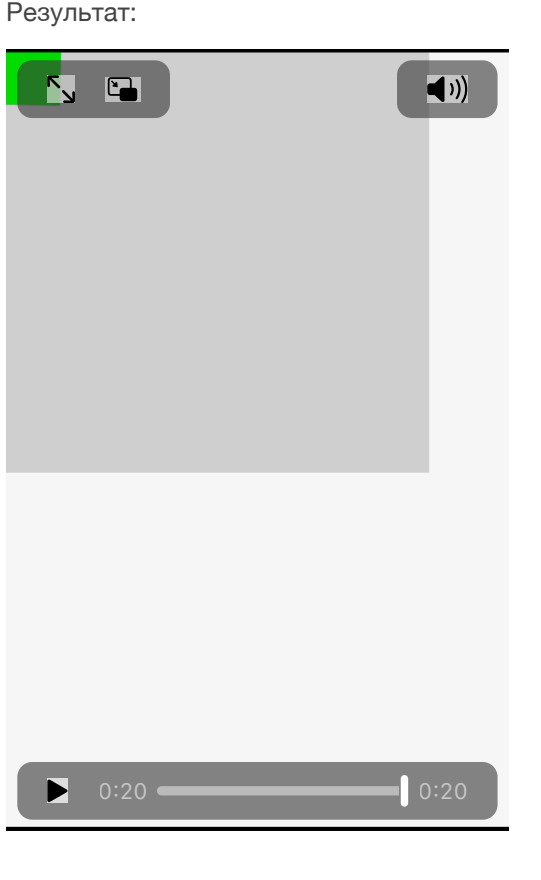
```
@Composable
fun HomeScreen() {
   val point = remember { mutableStateOf(Offset.Zero) }
   val pointSize = remember { mutableFloatStateOf(100F) }
   val pointDegrees = remember { mutableFloatStateOf(0F) }
   Column {
       Box(modifier = Modifier
           .size(300.dp)
           .background(Color.LightGray)
           .pointerInput(Unit) {
               awaitPointerEventScope {
                   while (true) {
                       val event = awaitPointerEvent()
                       val centroid = event.calculateCentroid()
                       point.value = centroid.takeIf { it != Offset.Unspecified } ?: Offset.Zero
                       val zoom = event.calculateZoom()
                       pointSize.floatValue *= zoom
                       val rotation = event.calculateRotation()
                       pointDegrees.floatValue += rotation
                   }
               }
           }
           .drawWithContent {
               drawContent()
               rotate(pointDegrees.floatValue, pivot = point.value) {
                   drawPoints(listOf(point.value), PointMode.Points, Color.Green, pointSize.floatVa
               }
           }
       )
   }
}
```

Добавляем pointSize, в котором храним размер квадрата. Этот размер будем менять в соответствии с данными из calculateZoom

Добавляем pointDegrees, в котором храним градус поворота квадрата. Этот размер будем менять в соответствии с данными из calculateRotation

Таким образом Zoom будет увеличивать/уменьшать квадрат, а Rotate будет его поворачивать.

Результат:



Присоединяйтесь к нам в **Telegram**:

- в канале StartAndroid публикуются ссылки на новые статьи с сайта startandroid.ru и интересные материалы с хабра, medium.com и т.п.

- в чатах решаем возникающие вопросы и проблемы по различным темам: Android, Compose, Kotlin, RxJava, Dagger, Тестирование, Performance

- ну и если просто хочется поговорить с коллегами по разработке, то есть чат Флудильня

Назад Вперёд

Добавить комментарий

Тема

Отправить

JComments

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора

При возникновении каких-либо вопросов, используйте форму обратной связи

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under MIT License.
Font Awesome font licensed under SIL OFL 1.1.